

Spring Boot Interview FAQ

Core concepts, commonly asked questions, and concise answers for backend developer interviews

1. Core Concepts

Q1. What is Spring Boot, and how is it different from the Spring Framework?

Spring Boot is built on top of the Spring Framework and removes the need for extensive manual configuration. It provides auto-configuration, embedded servers, starter dependencies, and production-ready features like Actuator, so a developer can go from zero to a running REST API with minimal XML or Java config, whereas plain Spring requires wiring beans and a servlet container explicitly.

Q2. What does `@SpringBootApplication` actually do?

It is a convenience annotation that combines three separate annotations: `@Configuration`, which marks the class as a source of bean definitions; `@EnableAutoConfiguration`, which tells Spring Boot to guess and configure beans based on the JARs present on the classpath; and `@ComponentScan`, which scans the current package and sub-packages for components, services, and repositories to register as beans.

Q3. How does Spring Boot auto-configuration work internally?

Auto-configuration classes are conditionally applied using annotations like `@ConditionalOnClass`, `@ConditionalOnMissingBean`, and `@ConditionalOnProperty`. At startup, Spring Boot scans `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` (spring.factories in older versions) for candidate auto-configuration classes and only activates each one if its conditions are satisfied, for example only configuring a `DataSource` bean if a JDBC driver is on the classpath and no `DataSource` bean already exists.

Q4. What are Spring Boot starters?

Starters are curated dependency descriptors that pull in a coherent set of libraries for a given purpose, so a developer does not have to resolve version compatibility manually. For example, `spring-boot-starter-web` brings in Spring MVC, an embedded Tomcat server, and Jackson for JSON, while `spring-boot-starter-data-jpa` brings in Hibernate and Spring Data JPA.

Q5. What is the difference between `application.properties` and `application.yml`?

Both configure the same underlying properties; YAML is more concise for hierarchical configuration and supports lists and nested maps more naturally, while properties files are flat key-value pairs. Spring Boot supports both simultaneously, though mixing them for the same keys in the same profile is discouraged since it makes precedence harder to reason about.

2. Dependency Injection and Bean Lifecycle

Q6. What are the different ways to inject a dependency in Spring?

Constructor injection, setter injection, and field injection using `@Autowired`. Constructor injection is the recommended approach because it allows the field to be declared final, makes the dependency explicit and mandatory, and makes the class easier to unit test without needing a Spring context to instantiate it.

Q7. What are the default and commonly used Spring bean scopes?

Singleton is the default: one shared instance per Spring container. Prototype creates a new instance every time the bean is requested. Request and session scopes, available in web applications, create one instance per HTTP request or per HTTP session respectively.

Q8. What is the difference between `@Component`, `@Service`, `@Repository`, and `@Controller`?

All four are stereotype annotations that register a class as a Spring bean via component scanning. They are functionally similar but signal intent: `@Repository` additionally enables automatic translation of persistence-layer exceptions into Spring's `DataAccessException` hierarchy, while `@Controller` and `@RestController` mark classes as MVC request handlers, with `@RestController` combining `@Controller` and `@ResponseBody` so return values are serialized directly into the response body.

Q9. What is circular dependency in Spring, and how can it be resolved?

A circular dependency occurs when Bean A depends on Bean B and Bean B depends back on Bean A. With field or setter injection, Spring can resolve this using a partially constructed bean reference, but with constructor injection it fails at startup, since neither bean can be fully constructed first. The usual fix is to redesign the beans to remove the cycle, or to break the cycle with setter injection or a `@Lazy` annotation on one of the dependencies.

3. REST APIs, Validation, and Exception Handling

Q10. How do you implement global exception handling in a Spring Boot REST API?

Using `@ControllerAdvice` combined with `@ExceptionHandler` methods. A single class annotated with `@ControllerAdvice` can intercept exceptions thrown from any controller, map them to an appropriate HTTP status code and a structured error response body, avoiding repetitive try-catch blocks in every controller method.

Q11. How does request body validation work with `@Valid`?

A DTO class is annotated with Bean Validation constraints such as `@NotNull`, `@Size`, or `@Email` from `jakarta.validation.constraints`. The controller method parameter is annotated with `@Valid`, and Spring automatically validates the incoming request body before the method executes, throwing a `MethodArgumentNotValidException` if validation fails, which can be caught by a `@ControllerAdvice` handler to return a 400 Bad Request with field-level error details.

Q12. What is the difference between `@PathVariable` and `@RequestParam`?

`@PathVariable` extracts a value from a segment of the URI path itself, such as the id in `/orders/{id}`, while `@RequestParam` extracts a value from the query string, such as `?status=PAID` in `/orders?status=PAID`.

Q13. What is HATEOAS, and is it commonly used in practice?

HATEOAS (Hypermedia as the Engine of Application State) means a REST response includes links to related actions or resources so a client can navigate the API dynamically rather than hardcoding URLs. Spring HATEOAS provides support for this, but in practice most internal or product-company APIs skip full HATEOAS compliance in favor of simpler, documented REST contracts, since the added complexity rarely pays off outside of public, evolving APIs.

4. Data Access and Transactions

Q14. What is the difference between JPA, Hibernate, and Spring Data JPA?

JPA is a specification, not an implementation, defining a standard set of interfaces for object-relational mapping. Hibernate is the most widely used implementation of that specification. Spring Data JPA is a Spring abstraction layer on top of JPA that eliminates boilerplate DAO code by generating repository implementations from interface method names or `@Query` annotations.

Q15. What does `@Transactional` actually do, and what is a common mistake with it?

It wraps the annotated method in a database transaction, committing on success and rolling back on a runtime exception by default (checked exceptions do not trigger a rollback unless explicitly configured). A very common mistake is calling a `@Transactional` method from another method within the same class: Spring's proxy-based AOP means the annotation is only honored when the method is called through the Spring-managed proxy from outside the class, so an internal self-invocation silently bypasses the transaction.

Q16. What is the N+1 select problem, and how do you avoid it?

It occurs when fetching a list of parent entities triggers one query for the parent list plus one additional query per parent to lazily fetch an associated collection, resulting in N extra queries. It is typically avoided using a JOIN FETCH clause in a JPQL query, an `@EntityGraph` annotation, or by switching the relevant association to eager fetching only where appropriate.

Q17. What is the difference between `FetchType.LAZY` and `FetchType.EAGER`?

LAZY defers loading an associated entity or collection until it is actually accessed in code, reducing the initial query cost but risking a `LazyInitializationException` if accessed outside an active persistence context. EAGER loads the association immediately as part of the initial query. `@ManyToOne` and `@OneToOne`

default to EAGER, while @OneToMany and @ManyToMany default to LAZY, and most teams override the eager defaults to LAZY explicitly for predictable performance.

5. Profiles, Testing, and Microservices Basics

Q18. What are Spring profiles, and how are they used?

Profiles let a single codebase carry environment-specific configuration and bean definitions, activated at startup via the `spring.profiles.active` property. A class or bean can be restricted to a profile using `@Profile("dev")`, and properties files named `application-dev.yml` or `application-prod.yml` are loaded in addition to the base `application.yml` when that profile is active.

Q19. What is Spring Boot Actuator, and what does it expose?

Actuator adds production-ready monitoring endpoints to an application, such as `/actuator/health` for liveness and readiness, `/actuator/metrics` for JVM and application metrics, `/actuator/env` for resolved configuration properties, and `/actuator/beans` for a full list of registered beans. Most endpoints beyond health are disabled by default in production and must be explicitly exposed and secured.

Q20. What is the difference between `@Mock`, `@MockBean`, and `@SpyBean` in tests?

`@Mock`, from Mockito directly, creates a mock object without any Spring context involvement, typically used in a plain unit test. `@MockBean` is a Spring Boot Test annotation that replaces the corresponding bean in the Spring application context with a mock, useful for slice tests like `@WebMvcTest` where the rest of the context still needs to load. `@SpyBean` similarly wraps a real bean from the context but allows selectively stubbing only certain method calls while letting others execute normally.

Q21. What is the difference between `@WebMvcTest`, `@DataJpaTest`, and `@SpringBootTest`?

`@WebMvcTest` loads only the web layer, mocking service and repository beans, and is used for fast controller-focused tests. `@DataJpaTest` loads only the JPA-related configuration against an in-memory or embedded database, used for repository tests. `@SpringBootTest` loads the full application context and is used for broader integration tests, at the cost of being significantly slower than the sliced test annotations.

Q22. How does Spring Boot support building microservices?

Spring Boot itself provides the embedded server, auto-configuration, and Actuator needed to run a lightweight, independently deployable service. The broader Spring Cloud project layers microservices-specific concerns on top, including Eureka for service discovery, Spring Cloud Gateway for routing, Spring Cloud Config for centralized configuration, and Resilience4j integration for circuit breakers and retries.

Q23. What is the difference between WebMVC and WebFlux?

Spring WebMVC is the traditional, synchronous, blocking servlet-based stack, where each request occupies a thread for its full duration. Spring WebFlux is the reactive, non-blocking stack built on Project Reactor and Netty, where a small number of threads handle many concurrent requests via an event loop, which suits high-concurrency, I/O-bound workloads better than thread-per-request but has a steeper learning curve and is not a drop-in replacement for existing blocking JDBC code.